

Back To Practice

< Q1 >

Save

Round Robin CPU Scheduling Algorithm

Write a C program to implement the Round Robin (RR) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), Burst Times (BT), and a Time Quantum (TQ). The CPU should rotate through the processes in the ready queue, granting each a maximum of one TQ of execution time. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and the overall averages.

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the Time Quantum.
3. The next lines contain two integers for each process, where
 - The first integer denotes the Arrival Time (AT)
 - The second integer denotes the Burst Time (BT) of the process.

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Sample Input 1

Select Language : C (GCC 9.2.0)

```
1 #include <stdio.h>
2
3 int main() {
4     int n, tq;
5
6     printf("Enter number of processes: \n");
7     scanf("%d", &n);
8
9     printf("Enter Time Quantum: \n");
10    scanf("%d", &tq);
11
12    int at[n], bt[n], rt[n], ct[n];
13    int tat[n], wt[n];
14
15
16    for (int i = 0; i < n; i++) {
17        printf("Enter AT and BT for P%d: \n", i + 1);
18        scanf("%d %d", &at[i], &bt[i]);
19        rt[i] = bt[i]; // remaining time
20    }
21
22    int time = 0, completed = 0;
23
24
25    while (completed < n) {
26        int done = 1;
27
```

Test Cases 2

Run Sample Cases

Run All Cases

Back To Practice

< Q1 >

Save

Round Robin CPU Scheduling Algorithm

Write a C program to implement the Round Robin (RR) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), Burst Times (BT), and a Time Quantum (TQ). The CPU should rotate through the processes in the ready queue, granting each a maximum of one TQ of execution time. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and the overall averages.

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the Time Quantum.
3. The next lines contain two integers for each process, where

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT) of the process.

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Sample Input 1

Select Language : C (GCC 9.2.0)

```
27
28     for (int i = 0; i < n; i++) {
29         if (rt[i] > 0 && at[i] <= time) {
30             done = 0;
31
32             if (rt[i] > tq) {
33                 time += tq;
34                 rt[i] -= tq;
35             } else {
36                 time += rt[i];
37                 ct[i] = time;
38                 rt[i] = 0;
39                 completed++;
40             }
41         }
42     }
43
44     if (done)
45         time++;
46 }
47
48 float total_tat = 0, total_wt = 0;
49
50
51 for (int i = 0; i < n; i++) {
52     tat[i] = ct[i] - at[i];
53 }
```

Test Cases 2

Run Sample Cases

Run All Cases

Back To Practice

Save

Round Robin CPU Scheduling Algorithm

Write a C program to implement the Round Robin (RR) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), Burst Times (BT), and a Time Quantum (TQ). The CPU should rotate through the processes in the ready queue, granting each a maximum of one TQ of execution time. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and the overall averages.

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the Time Quantum.
3. The next lines contain two integers for each process, where

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT) of the process.

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Sample Input 1

Completed

Select Language : C (GCC 9.2.0)

```

38         rt[i] = 0;
39         completed++;
40     }
41 }
42 }
43
44
45     if (done)
46         time++;
47 }
48
49 float total_tat = 0, total_wt = 0;
50
51
52 for (int i = 0; i < n; i++) {
53     tat[i] = ct[i] - at[i];
54     wt[i] = tat[i] - bt[i];
55
56     total_tat += tat[i];
57     total_wt += wt[i];
58 }
59
60 printf("\nAverage Turnaround Time: %.2f\n", total_tat / n);
61 printf("Average Waiting Time: %.2f\n", total_wt / n);
62
63 return 0;
64 }
```

Test Cases 2

Run Sample Cases

Run All Cases